

- *30. Find the flaw with the following “proof” that $a^n = 1$ for all nonnegative integers n , whenever a is a nonzero real number.

Basis Step: $a^0 = 1$ is true by the definition of a^0 .

Inductive Step: Assume that $a^j = 1$ for all nonnegative integers j with $j \leq k$. Then note that

$$a^{k+1} = \frac{a^k \cdot a^k}{a^{k-1}} = \frac{1 \cdot 1}{1} = 1.$$

- *31. Show that strong induction is a valid method of proof by showing that it follows from the well-ordering property.

32. Find the flaw with the following “proof” that every postage of three cents or more can be formed using just 3-cent and 4-cent stamps.

Basis Step: We can form postage of three cents with a single 3-cent stamp and we can form postage of four cents using a single 4-cent stamp.

Inductive Step: Assume that we can form postage of j cents for all nonnegative integers j with $j \leq k$ using just 3-cent and 4-cent stamps. We can then form postage of $k + 1$ cents by replacing one 3-cent stamp with a 4-cent stamp or by replacing two 4-cent stamps by three 3-cent stamps.

33. Show that we can prove that $P(n, k)$ is true for all pairs of positive integers n and k if we show

- $P(1, 1)$ is true and $P(n, k) \rightarrow [P(n + 1, k) \wedge P(n, k + 1)]$ is true for all positive integers n and k .
- $P(1, k)$ is true for all positive integers k , and $P(n, k) \rightarrow P(n + 1, k)$ is true for all positive integers n and k .
- $P(n, 1)$ is true for all positive integers n , and $P(n, k) \rightarrow P(n, k + 1)$ is true for all positive integers n and k .

34. Prove that $\sum_{j=1}^n j(j+1)(j+2) \cdots (j+k-1) = n(n+1)(n+2) \cdots (n+k)/(k+1)$ for all positive integers k and n . [Hint: Use a technique from Exercise 33.]

- *35. Show that if $a_1, a_2, \dots, a_n$ are n distinct real numbers, exactly $n - 1$ multiplications are used to compute the product of these n numbers no matter how parentheses are inserted into their product. [Hint: Use strong induction and consider the last multiplication.]
- *36. The well-ordering property can be used to show that there is a unique greatest common divisor of two positive integers. Let a and b be positive integers, and let S be

the set of positive integers of the form $as + bt$, where s and t are integers.

- Show that S is nonempty.
- Use the well-ordering property to show that S has a smallest element c .
- Show that if d is a common divisor of a and b , then d is a divisor of c .
- Show that $c \mid a$ and $c \mid b$. [Hint: First, assume that $c \nmid a$. Then $a = qc + r$, where $0 < r < c$. Show that $r \in S$, contradicting the choice of c .]
- Conclude from (c) and (d) that the greatest common divisor of a and b exists. Finish the proof by showing that this greatest common divisor is unique.

37. Let a be an integer and d be a positive integer. Show that the integers q and r with $a = dq + r$ and $0 \leq r < d$, which were shown to exist in Example 5, are unique.

38. Use mathematical induction to show that a rectangular checkerboard with an even number of cells and two squares missing, one white and one black, can be covered by dominoes.

- **39. Can you use the well-ordering property to prove the statement: “Every positive integer can be described using no more than fifteen English words”? Assume the words come from a particular dictionary of English. [Hint: Suppose that there are positive integers that cannot be described using no more than fifteen English words. By well ordering, the smallest positive integer that cannot be described using no more than fifteen English words would then exist.]

40. Use the well-ordering property to show that if x and y are real numbers with $x < y$, then there is a rational number r with $x < r < y$. [Hint: Use the Archimedean property, given in Appendix 1, to find a positive integer A with $A > 1/(y - x)$. Then show that there is a rational number r with denominator A between x and y by looking at the numbers $\lfloor x \rfloor + j/A$, where j is a positive integer.]

- *41. Show that the well-ordering property can be proved when the principle of mathematical induction is taken as an axiom.
- *42. Show that the principle of mathematical induction and strong induction are equivalent; that is, each can be shown to be valid from the other.
- *43. Show that we can prove the well-ordering property when we take strong induction as an axiom instead of taking the well-ordering property as an axiom.

5.3

Recursive Definitions and Structural Induction

5.3.1 Introduction

Sometimes it is difficult to define an object explicitly. However, it may be easy to define this object in terms of itself. This process is called **recursion**. For instance, the picture shown in Figure 1 is produced recursively. First, an original picture is given. Then a process of successively superimposing centered smaller pictures on top of the previous pictures is carried out.

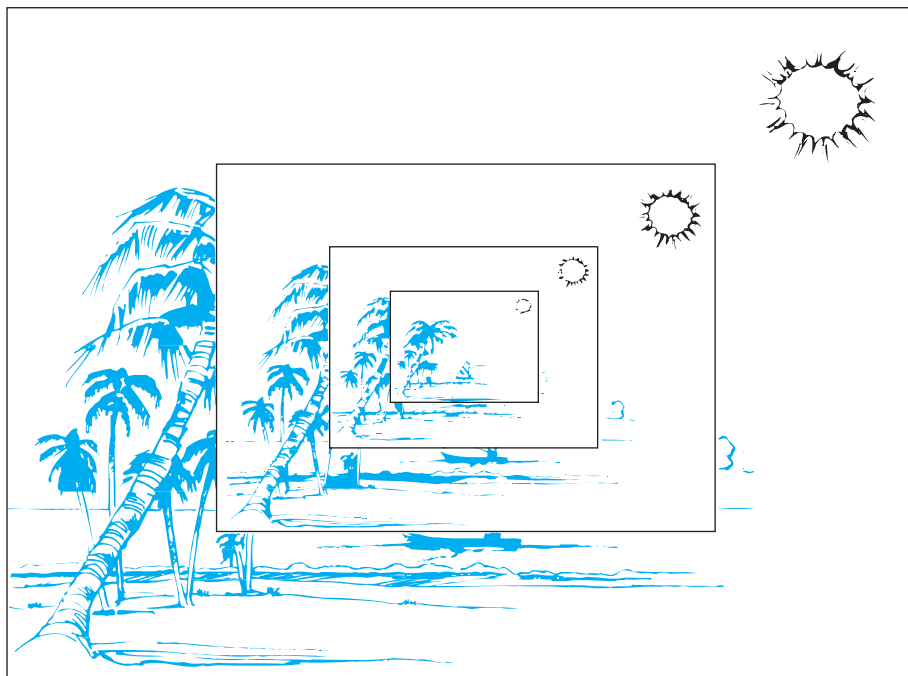


FIGURE 1 A recursively defined picture.

We can use recursion to define sequences, functions, and sets. In Section 2.4, and in most beginning mathematics courses, the terms of a sequence are specified using an explicit formula. For instance, the sequence of powers of 2 is given by $a_n = 2^n$ for $n = 0, 1, 2, \dots$. Recall from Section 2.4 that we can also define a sequence recursively by specifying how terms of the sequence are found from previous terms. The sequence of powers of 2 can also be defined by giving the first term of the sequence, namely, $a_0 = 1$, and a rule for finding a term of the sequence from the previous one, namely, $a_{n+1} = 2a_n$ for $n = 0, 1, 2, \dots$. When we define a sequence recursively by specifying how terms of the sequence are found from previous terms, we can use induction to prove results about the sequence.

When we define a set recursively, we specify some initial elements in a basis step and provide a rule for constructing new elements from those we already have in the recursive step. To prove results about recursively defined sets we use a method called *structural induction*.

5.3.2 Recursively Defined Functions

We use two steps to define a function with the set of nonnegative integers as its domain:

BASIS STEP: Specify the value of the function at zero.

RECURSIVE STEP: Give a rule for finding its value at an integer from its values at smaller integers.



Such a definition is called a **recursive** or **inductive definition**. Note that a function $f(n)$ from the set of nonnegative integers to the set of real numbers is the same as a sequence $a_0, a_1, \dots$, where a_i is a real number for every nonnegative integer i . So, defining a real-valued sequence $a_0, a_1, \dots$ using a recurrence relation, as was done in Section 2.4, is the same as defining a function from the set of nonnegative integers to the set of real numbers.

EXAMPLE 1 Suppose that f is defined recursively by

*Extra
Examples* 

$$\begin{aligned} f(0) &= 3, \\ f(n+1) &= 2f(n) + 3. \end{aligned}$$

Find $f(1)$, $f(2)$, $f(3)$, and $f(4)$.

Solution: From the recursive definition it follows that

$$\begin{aligned} f(1) &= 2f(0) + 3 = 2 \cdot 3 + 3 = 9, \\ f(2) &= 2f(1) + 3 = 2 \cdot 9 + 3 = 21, \\ f(3) &= 2f(2) + 3 = 2 \cdot 21 + 3 = 45, \\ f(4) &= 2f(3) + 3 = 2 \cdot 45 + 3 = 93. \end{aligned}$$


Recursively defined functions are **well defined**. That is, for every positive integer, the value of the function at this integer is determined in an unambiguous way. This means that given any positive integer, we can use the two parts of the definition to find the value of the function at that integer, and that we obtain the same value no matter how we apply the two parts of the definition. This is a consequence of the principle of mathematical induction. (See Exercise 58.) Additional examples of recursive definitions are given in Examples 2 and 3.

EXAMPLE 2 Give a recursive definition of a^n , where a is a nonzero real number and n is a nonnegative integer.

Solution: The recursive definition contains two parts. First a^0 is specified, namely, $a^0 = 1$. Then the rule for finding a^{n+1} from a^n , namely, $a^{n+1} = a \cdot a^n$, for $n = 0, 1, 2, 3, \dots$, is given. These two equations uniquely define a^n for all nonnegative integers n . 

EXAMPLE 3 Give a recursive definition of

$$\sum_{k=0}^n a_k.$$

Solution: The first part of the recursive definition is

$$\sum_{k=0}^0 a_k = a_0.$$

The second part is

$$\sum_{k=0}^{n+1} a_k = \left(\sum_{k=0}^n a_k \right) + a_{n+1}.$$


In some recursive definitions of functions, the values of the function at the first k positive integers are specified, and a rule is given for determining the value of the function at larger integers from its values at some or all of the preceding k integers. That recursive definitions defined in this way produce well-defined functions follows from strong induction (see Exercise 59).

Recall from Section 2.4 that the Fibonacci numbers, $f_0, f_1, f_2, \dots$, are defined by the equations $f_0 = 0, f_1 = 1$, and

Links 

$$f_n = f_{n-1} + f_{n-2}$$

for $n = 2, 3, 4, \dots$ [We can think of the Fibonacci number f_n either as the n th term of the sequence of Fibonacci numbers $f_0, f_1, \dots$ or as the value at the integer n of a function $f(n)$.]

We can use the recursive definition of the Fibonacci numbers to prove many properties of these numbers. We give one such property in Example 4.

EXAMPLE 4 Show that whenever $n \geq 3, f_n > \alpha^{n-2}$, where $\alpha = (1 + \sqrt{5})/2$.

Extra Examples 

Solution: We can use strong induction to prove this inequality. Let $P(n)$ be the statement $f_n > \alpha^{n-2}$. We want to show that $P(n)$ is true whenever n is an integer greater than or equal to 3.

BASIS STEP: First, note that

$$\alpha < 2 = f_3, \quad \alpha^2 = (3 + \sqrt{5})/2 < 3 = f_4,$$

so $P(3)$ and $P(4)$ are true.

INDUCTIVE STEP: Assume that $P(j)$ is true, namely, that $f_j > \alpha^{j-2}$, for all integers j with $3 \leq j \leq k$, where $k \geq 4$. We must show that $P(k+1)$ is true, that is, that $f_{k+1} > \alpha^{k-1}$. Because α is a solution of $x^2 - x - 1 = 0$ (as the quadratic formula shows), it follows that $\alpha^2 = \alpha + 1$. Therefore,

$$\alpha^{k-1} = \alpha^2 \cdot \alpha^{k-3} = (\alpha + 1)\alpha^{k-3} = \alpha \cdot \alpha^{k-3} + 1 \cdot \alpha^{k-3} = \alpha^{k-2} + \alpha^{k-3}.$$

By the inductive hypothesis, because $k \geq 4$, we have

$$f_{k-1} > \alpha^{k-3}, \quad f_k > \alpha^{k-2}.$$

Therefore, it follows that

$$f_{k+1} = f_k + f_{k-1} > \alpha^{k-2} + \alpha^{k-3} = \alpha^{k-1}.$$

Hence, $P(k+1)$ is true. This completes the proof. 

Remark: The inductive step of the proof by strong induction in Example 4 shows that whenever $k \geq 4$, $P(k+1)$ follows from the assumption that $P(j)$ is true for $3 \leq j \leq k$. Hence, the inductive step does *not* show that $P(3) \rightarrow P(4)$. Therefore, we had to show that $P(4)$ is true separately.

We can now show that the Euclidean algorithm, introduced in Section 4.3, uses $O(\log b)$ divisions to find the greatest common divisor of the positive integers a and b , where $a \geq b$.

THEOREM 1

LAMÉ'S THEOREM Let a and b be positive integers with $a \geq b$. Then the number of divisions used by the Euclidean algorithm to find $\gcd(a, b)$ is less than or equal to five times the number of decimal digits in b .

Proof: Recall that when the Euclidean algorithm is applied to find $\gcd(a, b)$ with $a \geq b$, this sequence of equations (where $a = r_0$ and $b = r_1$) is obtained.

$$\begin{aligned} r_0 &= r_1 q_1 + r_2 & 0 \leq r_2 < r_1, \\ r_1 &= r_2 q_2 + r_3 & 0 \leq r_3 < r_2, \\ &\vdots \\ r_{n-2} &= r_{n-1} q_{n-1} + r_n & 0 \leq r_n < r_{n-1}, \\ r_{n-1} &= r_n q_n. \end{aligned}$$

Here n divisions have been used to find $r_n = \gcd(a, b)$. Note that the quotients $q_1, q_2, \dots, q_{n-1}$ are all at least 1. Moreover, $q_n \geq 2$, because $r_n < r_{n-1}$. This implies that

$$\begin{aligned} r_n &\geq 1 = f_2, \\ r_{n-1} &\geq 2r_n \geq 2f_2 = f_3, \\ r_{n-2} &\geq r_{n-1} + r_n \geq f_3 + f_2 = f_4, \\ &\vdots \\ r_2 &\geq r_3 + r_4 \geq f_{n-1} + f_{n-2} = f_n, \\ b = r_1 &\geq r_2 + r_3 \geq f_n + f_{n-1} = f_{n+1}. \end{aligned}$$

It follows that if n divisions are used by the Euclidean algorithm to find $\gcd(a, b)$ with $a \geq b$, then $b \geq f_{n+1}$. By Example 4 we know that $f_{n+1} > \alpha^{n-1}$ for $n > 2$, where $\alpha = (1 + \sqrt{5})/2$. Therefore, it follows that $b > \alpha^{n-1}$. Furthermore, because $\log_{10} \alpha \approx 0.208 > 1/5$, we see that

$$\log_{10} b > (n-1) \log_{10} \alpha > (n-1)/5.$$

Hence, $n-1 < 5 \cdot \log_{10} b$. Now suppose that b has k decimal digits. Then $b < 10^k$ and $\log_{10} b < k$. It follows that $n-1 < 5k$, and because k is an integer, it follows that $n \leq 5k$. This finishes the proof. $\triangleleft$

Because the number of decimal digits in b , which equals $\lfloor \log_{10} b \rfloor + 1$, is less than or equal to $\log_{10} b + 1$, Theorem 1 tells us that the number of divisions required to find $\gcd(a, b)$ with

Links



©Mondadori Portfolio/
Hulton Fine Art
Collection/Getty Images

FIBONACCI (1170–1250) Fibonacci (short for *filius Bonacci*, or “son of Bonacci”) was also known as Leonardo of Pisa. He was born in the Italian commercial center of Pisa. Fibonacci was a merchant who traveled extensively throughout the Mideast, where he came into contact with Arabian mathematics. In his book *Liber Abaci*, Fibonacci introduced the European world to Arabic notation for numerals and algorithms for arithmetic. It was in this book that his well known rabbit problem (described in Section 8.1) appeared. Fibonacci also wrote books on geometry and trigonometry and on Diophantine equations, which involve finding integer solutions to equations.

$a > b$ is less than or equal to $5(\log_{10} b + 1)$. Because $5(\log_{10} b + 1)$ is $O(\log b)$, we see that $O(\log b)$ divisions are used by the Euclidean algorithm to find $\gcd(a, b)$ whenever $a > b$.

5.3.3 Recursively Defined Sets and Structures

Assessment

We have explored how functions can be defined recursively. We now turn our attention to how sets can be defined recursively. Just as in the recursive definition of functions, recursive definitions of sets have two parts, a **basis step** and a **recursive step**. In the basis step, an initial collection of elements is specified. In the recursive step, rules for forming new elements in the set from those already known to be in the set are provided. Recursive definitions may also include an **exclusion rule**, which specifies that a recursively defined set contains nothing other than those elements specified in the basis step or generated by applications of the recursive step. In our discussions, we will always tacitly assume that the exclusion rule holds and no element belongs to a recursively defined set unless it is in the initial collection specified in the basis step or can be generated using the recursive step one or more times. Later we will see how we can use a technique known as structural induction to prove results about recursively defined sets.

Examples 5, 6, 8, and 9 illustrate the recursive definition of sets. In each example, we show those elements generated by the first few applications of the recursive step.

EXAMPLE 5 Consider the subset S of the set of integers recursively defined by

BASIS STEP: $3 \in S$.

RECURSIVE STEP: If $x \in S$ and $y \in S$, then $x + y \in S$.

Extra Examples

The new elements found to be in S are 3 by the basis step, $3 + 3 = 6$ at the first application of the recursive step, $3 + 6 = 6 + 3 = 9$ and $6 + 6 = 12$ at the second application of the recursive step, and so on. We will show in Example 10 that S is the set of all positive multiples of 3. ◀

Recursive definitions play an important role in the study of strings. (See Chapter 13 for an introduction to the theory of formal languages, for example.) Recall from Section 2.4 that a string over an alphabet Σ is a finite sequence of symbols from Σ . We can define Σ^* , the set of strings over Σ , recursively, as Definition 1 shows.

Definition 1

The set Σ^* of *strings* over the alphabet Σ is defined recursively by

BASIS STEP: $\lambda \in \Sigma^*$ (where λ is the empty string containing no symbols).

RECURSIVE STEP: If $w \in \Sigma^*$ and $x \in \Sigma$, then $wx \in \Sigma^*$.

Links



©Paul Fearn/Alamy Stock Photo

GABRIEL LAMÉ (1795–1870) Gabriel Lamé entered the École Polytechnique in 1813, graduating in 1817. He continued his education at the École des Mines, graduating in 1820.

In 1820 Lamé went to Russia, where he was appointed director of the Schools of Highways and Transportation in St. Petersburg. Not only did he teach, but he also planned roads and bridges while in Russia. He returned to Paris in 1832, where he helped found an engineering firm. However, he soon left the firm, accepting the chair of physics at the École Polytechnique, which he held until 1844. While holding this position, he was active outside academia as an engineering consultant, serving as chief engineer of mines and participating in the building of railways.

Lamé contributed original work to number theory, applied mathematics, and thermodynamics. His best-known work involves the introduction of curvilinear coordinates. His work on number theory includes proving Fermat's last theorem for $n = 7$, as well as providing the upper bound for the number of divisions used by the Euclidean algorithm given in this text.

In the opinion of Gauss, one of the most important mathematicians of all time, Lamé was the foremost French mathematician of his time. However, French mathematicians considered him too practical, whereas French scientists considered him too theoretical.

The basis step of the recursive definition of strings says that the empty string belongs to Σ^* . The recursive step states that new strings are produced by adding a symbol from Σ to the end of strings in Σ^* . At each application of the recursive step, strings containing one additional symbol are generated.

EXAMPLE 6 If $\Sigma = \{0, 1\}$, the strings found to be in Σ^* , the set of all bit strings, are λ , specified to be in Σ^* in the basis step, 0 and 1 formed during the first application of the recursive step, 00, 01, 10, and 11 formed during the second application of the recursive step, and so on. ◀

Recursive definitions can be used to define operations or functions on the elements of recursively defined sets. This is illustrated in Definition 2 of the concatenation of two strings and Example 7 concerning the length of a string.

Definition 2

Two strings can be combined via the operation of *concatenation*. Let Σ be a set of symbols and Σ^* the set of strings formed from symbols in Σ . We can define the concatenation of two strings, denoted by $\cdot$, recursively as follows.

BASIS STEP: If $w \in \Sigma^*$, then $w \cdot \lambda = w$, where λ is the empty string.

RECURSIVE STEP: If $w_1 \in \Sigma^*$ and $w_2 \in \Sigma^*$ and $x \in \Sigma$, then $w_1 \cdot (w_2x) = (w_1 \cdot w_2)x$.

The concatenation of the strings w_1 and w_2 is often written as w_1w_2 rather than $w_1 \cdot w_2$. By repeated application of the recursive definition, it follows that the concatenation of two strings w_1 and w_2 consists of the symbols in w_1 followed by the symbols in w_2 . For instance, the concatenation of $w_1 = abra$ and $w_2 = cadabra$ is $w_1w_2 = abracadabra$.

EXAMPLE 7 Length of a String Give a recursive definition of $l(w)$, the length of the string w .

Solution: The length of a string can be recursively defined by

$$\begin{aligned} l(\lambda) &= 0; \\ l(wx) &= l(w) + 1 \text{ if } w \in \Sigma^* \text{ and } x \in \Sigma. \end{aligned}$$

Another important use of recursive definitions is to define **well-formed formulae** of various types. This is illustrated in Examples 8 and 9.

EXAMPLE 8 Well-Formed Formulae in Propositional Logic We can define the set of well-formed formulae in propositional logic involving **T**, **F**, propositional variables, and operators from the set $\{\neg, \wedge, \vee, \rightarrow, \leftrightarrow\}$.

BASIS STEP: **T**, **F**, and s , where s is a propositional variable, are well-formed formulae.

RECURSIVE STEP: If E and F are well-formed formulae, then $(\neg E)$, $(E \wedge F)$, $(E \vee F)$, $(E \rightarrow F)$, and $(E \leftrightarrow F)$ are well-formed formulae.

For example, by the basis step we know that **T**, **F**, p , and q are well-formed formulae, where p and q are propositional variables. From an initial application of the recursive step, we know that $(p \vee q)$, $(p \rightarrow \mathbf{F})$, $(\mathbf{F} \rightarrow q)$, and $(q \wedge \mathbf{F})$ are well-formed formulae. A second application of the recursive step shows that $((p \vee q) \rightarrow (q \wedge \mathbf{F}))$, $(q \vee (p \vee q))$, and $((p \rightarrow \mathbf{F}) \rightarrow \mathbf{T})$ are well-formed formulae. We leave it to the reader to show that $p\neg \wedge q$, $pq\wedge$, and $\neg \wedge pq$ are *not* well-formed formulae, by showing that none can be obtained using the basis step and one or more applications of the recursive step. ◀

EXAMPLE 9 Well-Formed Formulae of Operators and Operands We can define the set of well-formed formulae consisting of variables, numerals, and operators from the set $\{+, -, *, /, \uparrow\}$ (where $*$ denotes multiplication and $\uparrow$ denotes exponentiation) recursively.

BASIS STEP: x is a well-formed formula if x is a numeral or a variable.

RECURSIVE STEP: If F and G are well-formed formulae, then $(F + G)$, $(F - G)$, $(F * G)$, (F/G) , and $(F \uparrow G)$ are well-formed formulae.

For example, by the basis step we see that x , y , 0 , and 3 are well-formed formulae (as is any variable or numeral). Well-formed formulae generated by applying the recursive step once include $(x + 3)$, $(3 + y)$, $(x - y)$, $(3 - 0)$, $(x * 3)$, $(3 * y)$, $(3/0)$, (x/y) , $(3 \uparrow x)$, and $(0 \uparrow 3)$. Applying the recursive step twice shows that formulae such as $((x + 3) + 3)$ and $(x - (3 * y))$ are well-formed formulae. [Note that $(3/0)$ is a well-formed formula because we are concerned only with syntax matters here.] We leave it to the reader to show that each of the formulae $x3 +$, $y * + x$, and $* x/y$ is *not* a well-formed formula by showing that none of them can be obtained from the basis step and one or more applications of the recursive step. ◀

We will study trees extensively in Chapter 11. A tree is a special type of a graph; a graph is made up of vertices and edges connecting some pairs of vertices. We will study graphs in Chapter 10. We will briefly introduce them here to illustrate how they can be defined recursively.

Definition 3

The set of *rooted trees*, where a rooted tree consists of a set of vertices containing a distinguished vertex called the *root*, and edges connecting these vertices, can be defined recursively by these steps:

BASIS STEP: A single vertex r is a rooted tree.

RECURSIVE STEP: Suppose that $T_1, T_2, \dots, T_n$ are disjoint rooted trees with roots $r_1, r_2, \dots, r_n$, respectively. Then the graph formed by starting with a root r , which is not in any of the rooted trees $T_1, T_2, \dots, T_n$, and adding an edge from r to each of the vertices $r_1, r_2, \dots, r_n$, is also a rooted tree.

In Figure 2 we illustrate some of the rooted trees formed starting with the basis step and applying the recursive step one time and two times. Note that infinitely many rooted trees are formed at each application of the recursive definition.

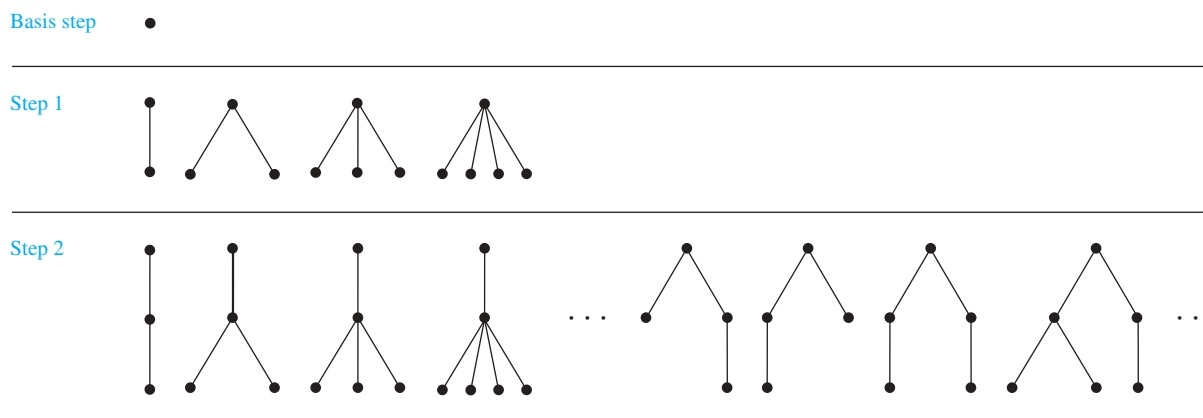


FIGURE 2 Building up rooted trees.

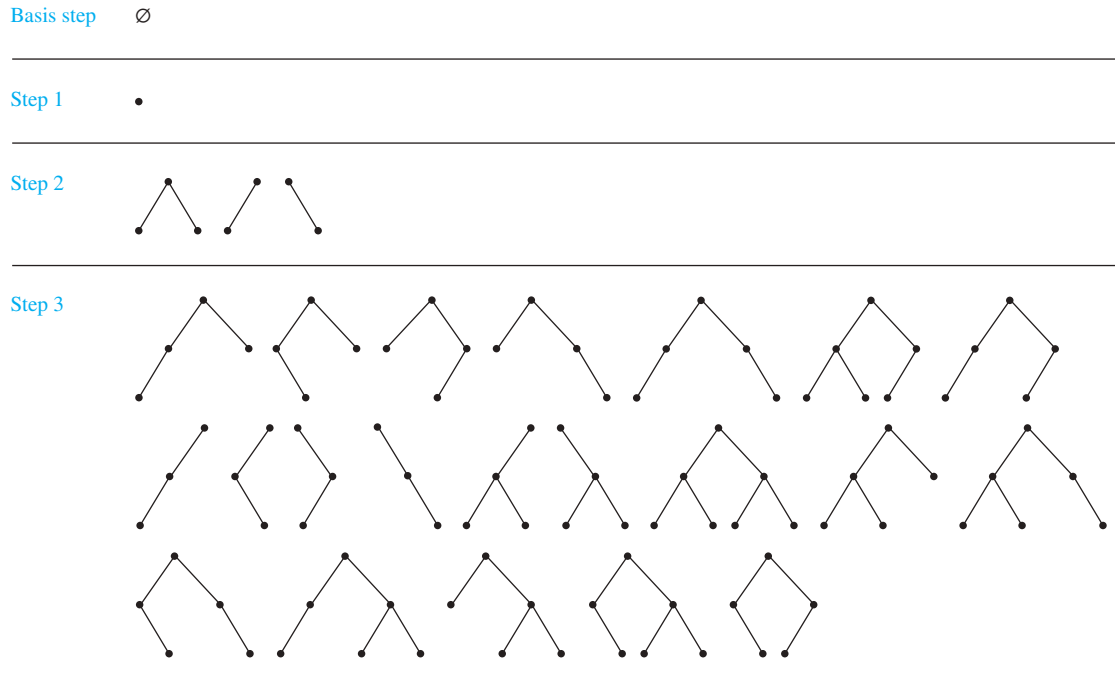


FIGURE 3 Building up extended binary trees.

Binary trees are a special type of rooted trees. We will provide recursive definitions of two types of binary trees—full binary trees and extended binary trees. In the recursive step of the definition of each type of binary tree, two binary trees are combined to form a new tree with one of these trees designated the left subtree and the other the right subtree. In extended binary trees, the left subtree or the right subtree can be empty, but in full binary trees this is not possible. Binary trees are one of the most important types of structures in computer science. In Chapter 11 we will see how they can be used in searching and sorting algorithms, in algorithms for compressing data, and in many other applications. We first define extended binary trees.

Definition 4

The set of *extended binary trees* can be defined recursively by these steps:

BASIS STEP: The empty set is an extended binary tree.

RECURSIVE STEP: If T_1 and T_2 are disjoint extended binary trees, there is an extended binary tree, denoted by $T_1 \cdot T_2$, consisting of a root r together with edges connecting the root to each of the roots of the left subtree T_1 and the right subtree T_2 when these trees are nonempty.

Figure 3 shows how extended binary trees are built up by applying the recursive step from one to three times.

We now show how to define the set of full binary trees. Note that the difference between this recursive definition and that of extended binary trees lies entirely in the basis step.

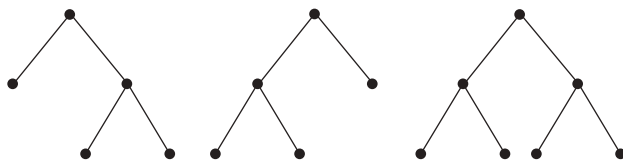
Basis step



Step 1



Step 2

**FIGURE 4** Building up full binary trees.**Definition 5**

The set of full binary trees can be defined recursively by these steps:

BASIS STEP: There is a full binary tree consisting only of a single vertex r .

RECURSIVE STEP: If T_1 and T_2 are disjoint full binary trees, there is a full binary tree, denoted by $T_1 \cdot T_2$, consisting of a root r together with edges connecting the root to each of the roots of the left subtree T_1 and the right subtree T_2 .

Figure 4 shows how full binary trees are built up by applying the recursive step one and two times.

5.3.4 Structural Induction

To prove results about recursively defined sets, we generally use some form of mathematical induction. Example 10 illustrates the connection between recursively defined sets and mathematical induction.

EXAMPLE 10 Show that the set S defined in Example 5 by specifying that $3 \in S$ and that if $x \in S$ and $y \in S$, then $x + y \in S$, is the set of all positive integers that are multiples of 3.

Solution: Let A be the set of all positive integers divisible by 3. To prove that $A = S$, we must show that A is a subset of S and that S is a subset of A . To prove that A is a subset of S , we must show that every positive integer divisible by 3 is in S . We will use mathematical induction to prove this.

Let $P(n)$ be the statement that $3n$ belongs to S . The basis step holds because by the first part of the recursive definition of S , $3 \cdot 1 = 3$ is in S . To establish the inductive step, assume that $P(k)$ is true, namely, that $3k$ is in S . Because $3k$ is in S and because 3 is in S , it follows from the second part of the recursive definition of S that $3k + 3 = 3(k + 1)$ is also in S .

To prove that S is a subset of A , we use the recursive definition of S . First, the basis step of the definition specifies that 3 is in S . Because $3 = 3 \cdot 1$, all elements specified to be in S in this step are divisible by 3 and are therefore in A . To finish the proof, we must show that all integers in S generated using the second part of the recursive definition are in A . This consists of showing that $x + y$ is in A whenever x and y are elements of S also assumed to be in A . Now if x and y are both in A , it follows that $3 \mid x$ and $3 \mid y$. By part (i) of Theorem 1 of Section 4.1, it follows that $3 \mid (x + y)$ completing the proof. ◀

In Example 10 we used mathematical induction over the set of positive integers and a recursive definition to prove a result about a recursively defined set. However, instead of using mathematical induction directly to prove results about recursively defined sets, we can use a more convenient form of induction known as **structural induction**. A proof by structural induction consists of two parts. These parts are

BASIS STEP: Show that the result holds for all elements specified in the basis step of the recursive definition to be in the set.

RECURSIVE STEP: Show that if the statement is true for each of the elements used to construct new elements in the recursive step of the definition, the result holds for these new elements.

The validity of structural induction follows from the principle of mathematical induction for the nonnegative integers. To see this, let $P(n)$ state that the claim is true for all elements of the set that are generated by n or fewer applications of the rules in the recursive step of a recursive definition. We will have established that the principle of mathematical induction implies the principle of structural induction if we can show that $P(n)$ is true whenever n is a positive integer. In the basis step of a proof by structural induction we show that $P(0)$ is true. That is, we show that the result is true of all elements specified to be in the set in the basis step of the definition. A consequence of the recursive step is that if we assume $P(k)$ is true, it follows that $P(k + 1)$ is true. When we have completed a proof using structural induction, we have shown that $P(0)$ is true and that $P(k)$ implies $P(k + 1)$. By mathematical induction it follows that $P(n)$ is true for all nonnegative integers n . This also shows that the result is true for all elements generated by the recursive definition, and shows that structural induction is a valid proof technique.

EXAMPLES OF PROOFS USING STRUCTURAL INDUCTION Structural induction can be used to prove that all members of a set constructed recursively have a particular property. We will illustrate this idea by using structural induction to prove results about well-formed formulae, strings, and binary trees. For each proof, we have to carry out the appropriate basis step and the appropriate recursive step. For example, to use structural induction to prove a result about the set of well-formed formulae defined in Example 8, where we specify that **T**, **F**, and every propositional variable s are well-formed formulae and where we specify that if E and F are well-formed formulae, then $(\neg E)$, $(E \wedge F)$, $(E \vee F)$, $(E \rightarrow F)$, and $(E \leftrightarrow F)$ are well-formed formulae, we need to complete this basis step and this recursive step.

BASIS STEP: Show that the result is true for **T**, **F**, and s whenever s is a propositional variable.

RECURSIVE STEP: Show that if the result is true for the compound propositions p and q , it is also true for $(\neg p)$, $(p \vee q)$, $(p \wedge q)$, $(p \rightarrow q)$, and $(p \leftrightarrow q)$.

Example 11 illustrates how we can prove results about well-formed formulae using structural induction.

EXAMPLE 11 Show that every well-formed formula for compound propositions, as defined in Example 8, contains an equal number of left and right parentheses.

Solution:

BASIS STEP: Each of the formula **T**, **F**, and s contains no parentheses, so clearly they contain an equal number of left and right parentheses.

RECURSIVE STEP: Assume p and q are well-formed formulae, each containing an equal number of left and right parentheses. That is, if l_p and l_q are the number of left parentheses in p and q , respectively, and r_p and r_q are the number of right parentheses in p and q , respectively, then $l_p = r_p$ and $l_q = r_q$. To complete the inductive step, we need to show that each of $(\neg p)$, $(p \vee q)$, $(p \wedge q)$, $(p \rightarrow q)$, and $(p \leftrightarrow q)$ also contains an equal number of left and right parentheses. The number of left parentheses in the first of these compound propositions equals

$l_p + 1$ and in each of the other compound propositions equals $l_p + l_q + 1$. Similarly, the number of right parentheses in the first of these compound propositions equals $r_p + 1$ and in each of the other compound propositions equals $r_p + r_q + 1$. Because $l_p = r_p$ and $l_q = r_q$, it follows that each of these compound expressions contains the same number of left and right parentheses. This completes the proof by structural induction. 

Suppose that $P(w)$ is a propositional function over the set of strings $w \in \Sigma^*$. To use structural induction to prove that $P(w)$ holds for all strings $w \in \Sigma^*$, we need to complete both a basis step and a recursive step. These steps are:

BASIS STEP: Show that $P(\lambda)$ is true.

RECURSIVE STEP: Assume that $P(w)$ is true, where $w \in \Sigma^*$. Show that if $x \in \Sigma$, then $P(wx)$ must also be true.

Example 12 illustrates how structural induction can be used in proofs about strings.

EXAMPLE 12 Use structural induction to prove that $l(xy) = l(x) + l(y)$, where x and y belong to Σ^* , the set of strings over the alphabet Σ .

Solution: We will base our proof on the recursive definition of the set Σ^* given in Definition 1 and the definition of the length of a string in Example 7, which specifies that $l(\lambda) = 0$ and $l(wx) = l(w) + 1$ when $w \in \Sigma^*$ and $x \in \Sigma$. Let $P(y)$ be the statement that $l(xy) = l(x) + l(y)$ whenever x belongs to Σ^* .

BASIS STEP: To complete the basis step, we must show that $P(\lambda)$ is true. That is, we must show that $l(x\lambda) = l(x) + l(\lambda)$ for all $x \in \Sigma^*$. Because $l(x\lambda) = l(x) = l(x) + 0 = l(x) + l(\lambda)$ for every string x , it follows that $P(\lambda)$ is true.

RECURSIVE STEP: To complete the inductive step, we assume that $P(y)$ is true and show that this implies that $P(ya)$ is true whenever $a \in \Sigma$. What we need to show is that $l(xya) = l(x) + l(ya)$ for every $a \in \Sigma$. To show this, note that by the recursive definition of $l(w)$ (given in Example 7), we have $l(xya) = l(xy) + 1$ and $l(ya) = l(y) + 1$. And, by the inductive hypothesis, $l(xy) = l(x) + l(y)$. We conclude that $l(xya) = l(x) + l(y) + 1 = l(x) + l(ya)$. 

We can prove results about trees or special classes of trees using structural induction. For example, to prove a result about full binary trees using structural induction we need to complete this basis step and this recursive step.

BASIS STEP: Show that the result is true for the tree consisting of a single vertex.

RECURSIVE STEP: Show that if the result is true for the full binary trees T_1 and T_2 , then it is true for tree $T_1 \cdot T_2$ consisting of a root r , which has T_1 as its left subtree and T_2 as its right subtree.

Before we provide an example showing how structural induction can be used to prove a result about full binary trees, we need some definitions. We will recursively define the height $h(T)$ and the number of vertices $n(T)$ of a full binary tree T . We begin by defining the height of a full binary tree.

Definition 6

We define the height $h(T)$ of a full binary tree T recursively.

BASIS STEP: The height of the full binary tree T consisting of only a root r is $h(T) = 0$.

RECURSIVE STEP: If T_1 and T_2 are full binary trees, then the full binary tree $T = T_1 \cdot T_2$ has height $h(T) = 1 + \max(h(T_1), h(T_2))$.

If we let $n(T)$ denote the number of vertices in a full binary tree, we observe that $n(T)$ satisfies the following recursive formula:

BASIS STEP: The number of vertices $n(T)$ of the full binary tree T consisting of only a root r is $n(T) = 1$.

RECURSIVE STEP: If T_1 and T_2 are full binary trees, then the number of vertices of the full binary tree $T = T_1 \cdot T_2$ is $n(T) = 1 + n(T_1) + n(T_2)$.

We now show how structural induction can be used to prove a result about full binary trees.

THEOREM 2

If T is a full binary tree T , then $n(T) \leq 2^{h(T)+1} - 1$.

Proof: We prove this inequality using structural induction.

BASIS STEP: For the full binary tree consisting of just the root r the result is true because $n(T) = 1$ and $h(T) = 0$, so that $n(T) = 1 \leq 2^{0+1} - 1 = 1$.

RECURSIVE STEP: For the inductive hypothesis we assume that $n(T_1) \leq 2^{h(T_1)+1} - 1$ and $n(T_2) \leq 2^{h(T_2)+1} - 1$ whenever T_1 and T_2 are full binary trees. By the recursive formulae for $n(T)$ and $h(T)$ we have $n(T) = 1 + n(T_1) + n(T_2)$ and $h(T) = 1 + \max(h(T_1), h(T_2))$.

We find that

$$\begin{aligned}
 n(T) &= 1 + n(T_1) + n(T_2) && \text{by the recursive formula for } n(T) \\
 &\stackrel{\text{IH}}{\leq} 1 + (2^{h(T_1)+1} - 1) + (2^{h(T_2)+1} - 1) && \text{by the inductive hypothesis} \\
 &\leq 2 \cdot \max(2^{h(T_1)+1}, 2^{h(T_2)+1}) - 1 && \text{because the sum of two terms is at most 2} \\
 &\quad \text{times the larger} \\
 &= 2 \cdot 2^{\max(h(T_1), h(T_2))+1} - 1 && \text{because } \max(2^x, 2^y) = 2^{\max(x,y)} \\
 &= 2 \cdot 2^{h(T)} - 1 && \text{by the recursive definition of } h(T) \\
 &= 2^{h(T)+1} - 1.
 \end{aligned}$$

This completes the recursive step. ◀

5.3.5 Generalized Induction

We can extend mathematical induction to prove results about other sets that have the well-ordering property besides the set of integers. Although we will discuss this concept in detail in Section 9.6, we provide an example here to illustrate the usefulness of such an approach.

As an example, note that we can define an ordering on $\mathbf{N} \times \mathbf{N}$, the ordered pairs of non-negative integers, by specifying that (x_1, y_1) is less than or equal to (x_2, y_2) if either $x_1 < x_2$, or $x_1 = x_2$ and $y_1 < y_2$; this is called the **lexicographic ordering**. The set $\mathbf{N} \times \mathbf{N}$ with this ordering has the property that every subset of $\mathbf{N} \times \mathbf{N}$ has a least element (see Exercise 53 in Section 9.6). This implies that we can recursively define the terms $a_{m,n}$, with $m \in \mathbf{N}$ and $n \in \mathbf{N}$, and prove results about them using a variant of mathematical induction, as illustrated in Example 13.

EXAMPLE 13 Suppose that $a_{m,n}$ is defined recursively for $(m, n) \in \mathbf{N} \times \mathbf{N}$ by $a_{0,0} = 0$ and

$$a_{m,n} = \begin{cases} a_{m-1,n} + 1 & \text{if } n = 0 \text{ and } m > 0 \\ a_{m,n-1} + n & \text{if } n > 0. \end{cases}$$

Show that $a_{m,n} = m + n(n+1)/2$ for all $(m, n) \in \mathbf{N} \times \mathbf{N}$, that is, for all pairs of nonnegative integers.

Solution: We can prove that $a_{m,n} = m + n(n+1)/2$ using a generalized version of mathematical induction. The basis step requires that we show that this formula is valid when $(m, n) = (0, 0)$. The induction step requires that we show that if the formula holds for all pairs smaller than (m, n) in the lexicographic ordering of $\mathbf{N} \times \mathbf{N}$, then it also holds for (m, n) .

BASIS STEP: Let $(m, n) = (0, 0)$. Then by the basis case of the recursive definition of $a_{m,n}$ we have $a_{0,0} = 0$. Furthermore, when $m = n = 0$, $m + n(n+1)/2 = 0 + (0 \cdot 1)/2 = 0$. This completes the basis step.

INDUCTIVE STEP: Suppose that $a_{m',n'} = m' + n'(n'+1)/2$ whenever (m', n') is less than (m, n) in the lexicographic ordering of $\mathbf{N} \times \mathbf{N}$. By the recursive definition, if $n = 0$, then $a_{m,n} = a_{m-1,n} + 1$. Because $(m-1, n)$ is smaller than (m, n) , the inductive hypothesis tells us that $a_{m-1,n} = m-1 + n(n+1)/2$, so that $a_{m,n} = m-1 + n(n+1)/2 + 1 = m + n(n+1)/2$, giving us the desired equality. Now suppose that $n > 0$, so $a_{m,n} = a_{m,n-1} + n$. Because $(m, n-1)$ is smaller than (m, n) , the inductive hypothesis tells us that $a_{m,n-1} = m + (n-1)n/2$, so $a_{m,n} = m + (n-1)n/2 + n = m + (n^2 - n + 2n)/2 = m + n(n+1)/2$. This finishes the inductive step. 

As mentioned, we will justify this proof technique in Section 9.6.

Exercises

- Find $f(1), f(2), f(3)$, and $f(4)$ if $f(n)$ is defined recursively by $f(0) = 1$ and for $n = 0, 1, 2, \dots$
 - $f(n+1) = f(n) + 2$.
 - $f(n+1) = 3f(n)$.
 - $f(n+1) = 2^{f(n)}$.
 - $f(n+1) = f(n)^2 + f(n) + 1$.
- Find $f(1), f(2), f(3), f(4)$, and $f(5)$ if $f(n)$ is defined recursively by $f(0) = 3$ and for $n = 0, 1, 2, \dots$
 - $f(n+1) = -2f(n)$.
 - $f(n+1) = 3f(n) + 7$.
 - $f(n+1) = f(n)^2 - 2f(n) - 2$.
 - $f(n+1) = 3^{f(n)/3}$.
- Find $f(2), f(3), f(4)$, and $f(5)$ if f is defined recursively by $f(0) = -1, f(1) = 2$, and for $n = 1, 2, \dots$
 - $f(n+1) = f(n) + 3f(n-1)$.
 - $f(n+1) = f(n)^2 f(n-1)$.
 - $f(n+1) = 3f(n)^2 - 4f(n-1)^2$.
 - $f(n+1) = f(n-1)/f(n)$.
- Find $f(2), f(3), f(4)$, and $f(5)$ if f is defined recursively by $f(0) = f(1) = 1$ and for $n = 1, 2, \dots$
 - $f(n+1) = f(n) - f(n-1)$.
 - $f(n+1) = f(n)f(n-1)$.
 - $f(n+1) = f(n)^2 + f(n-1)^3$.
 - $f(n+1) = f(n)/f(n-1)$.
- Determine whether each of these proposed definitions is a valid recursive definition of a function f from the set of nonnegative integers to the set of integers. If f is well defined, find a formula for $f(n)$ when n is a nonnegative integer and prove that your formula is valid.
 - $f(0) = 0, f(n) = 2f(n-2)$ for $n \geq 1$
 - $f(0) = 1, f(n) = f(n-1) - 1$ for $n \geq 1$
 - $f(0) = 2, f(1) = 3, f(n) = f(n-1) - 1$ for $n \geq 2$
 - $f(0) = 1, f(1) = 2, f(n) = 2f(n-2)$ for $n \geq 2$
 - $f(0) = 1, f(n) = 3f(n-1)$ if n is odd and $n \geq 1$ and $f(n) = 9f(n-2)$ if n is even and $n \geq 2$
- Determine whether each of these proposed definitions is a valid recursive definition of a function f from the set of nonnegative integers to the set of integers. If f is well defined, find a formula for $f(n)$ when n is a nonnegative integer and prove that your formula is valid.
 - $f(0) = 1, f(n) = -f(n-1)$ for $n \geq 1$
 - $f(0) = 1, f(1) = 0, f(2) = 2, f(n) = 2f(n-3)$ for $n \geq 3$
 - $f(0) = 0, f(1) = 1, f(n) = 2f(n+1)$ for $n \geq 2$
 - $f(0) = 0, f(1) = 1, f(n) = 2f(n-1)$ for $n \geq 1$
 - $f(0) = 2, f(n) = f(n-1)$ if n is odd and $n \geq 1$ and $f(n) = 2f(n-2)$ if $n \geq 2$

8. Give a recursive definition of the sequence $\{a_n\}$, $n = 1, 2, 3, \dots$ if

9. Let F be the function such that $F(n)$ is the sum of the first n positive integers. Give a recursive definition of $F(n)$.

10. Give a recursive definition of $S_m(n)$, the sum of the integer m and the nonnegative integer n .

- 11.** Give a recursive definition of $P_m(n)$, the product of the integer m and the nonnegative integer n .

12. Prove that $f_1^2 + f_2^2 + \cdots + f_n^2 = f_n f_{n+1}$ when n is a positive integer.

13. Prove that $f_1 + f_3 + \cdots + f_{2n-1} = f_{2n}$ when n is a positive integer.

- *14.** Show that $f_{n+1}f_{n-1} - f_n^2 = (-1)^n$ when n is a positive integer.

- *15. Show that $f_0f_1 + f_1f_2 + \cdots + f_{2n-1}f_{2n} = f_{2n}^2$ when n is a positive integer.

- *16. Show that $f_0 - f_1 + f_2 - \cdots - f_{2n-1} + f_{2n} = f_{2n-1} - 1$ when n is a positive integer.

17. Determine the number of divisions used by the Euclidean algorithm to find the greatest common divisor of the Fibonacci numbers f_n and f_{n+1} , where n is a nonnegative integer. Verify your answer using mathematical induction.

- $$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}.$$

$$\mathbf{A}^n = \begin{bmatrix} f_{n+1} & f_n \\ f_n & f_{n-1} \end{bmatrix}$$

19. By taking determinants of both sides of the equation in Exercise 18, prove the identity given in Exercise 14. (Recall that the determinant of the matrix $\begin{vmatrix} a & b \\ c & d \end{vmatrix}$ is $ad - bc$.)

- *20. Give a recursive definition of the functions $\max$ and $\min$ so that $\max(a_1, a_2, \dots, a_n)$ and $\min(a_1, a_2, \dots, a_n)$ are the maximum and minimum of the n numbers $a_1, a_2, \dots, a_n$, respectively.

- *21.** Let $a_1, a_2, \dots, a_n$, and $b_1, b_2, \dots, b_n$ be real numbers. Use the recursive definitions that you gave in Exercise 20 to prove these.

- a) $\max(-a_1, -a_2, \dots, -a_n) = -\min(a_1, a_2, \dots, a_n)$
- b) $\max(a_1 + b_1, a_2 + b_2, \dots, a_n + b_n)$
 $\leq \max(a_1, a_2, \dots, a_n) + \max(b_1, b_2, \dots, b_n)$
- c) $\min(a_1 + b_1, a_2 + b_2, \dots, a_n + b_n)$
 $\geq \min(a_1, a_2, \dots, a_n) + \min(b_1, b_2, \dots, b_n)$

- 22.** Show that the set S defined by $1 \in S$ and $s + t \in S$ whenever $s \in S$ and $t \in S$ is the set of positive integers.

- 23.** Give a recursive definition of the set of positive integers that are multiples of 5.

- a) the set of odd positive integers.
- b) the set of positive integer powers of 3.
- c) the set of polynomials with integer coefficients.

- the set of even integers.
- the set of positive integers congruent to 2 modulo 3.
- the set of positive integers not divisible by 5.

- Basis step:* $1 \in S$.

Recursive step: If $n \in S$, then $3n + 2 \in S$ and $n^2 \in S$.

- Show that if $n \in S$, then $n \equiv 1 \pmod{4}$.
- Show that there exists an integer $m \equiv 1 \pmod{4}$ that does not belong to S .

27. Let S be the set of positive integers defined by

Basis step: $5 \in S$.

Recursive step: If $n \in S$, then $3n \in S$ and $n^2 \in S$.

- Show that if $n \in S$, then $n \equiv 5 \pmod{10}$.
- Show that there exists an integer $m \equiv 5 \pmod{10}$ that does not belong to S .

28. Let S be the subset of the set of ordered pairs of integers defined recursively by

Basis step: $(0, 0) \in S$.

Recursive step: If $(a, b) \in S$, then $(a + 2, b + 3) \in S$ and $(a + 3, b + 2) \in S$.

- List the elements of S produced by the first five applications of the recursive definition.
- Use strong induction on the number of applications of the recursive step of the definition to show that $5 \mid a + b$ when $(a, b) \in S$.
- Use structural induction to show that $5 \mid a + b$ when $(a, b) \in S$.

- 29.** Let S be the subset of the set of ordered pairs of integers defined recursively by

Basis step: $(0, 0) \in S$.

Recursive step: If $(a, b) \in S$, then $(a, b + 1) \in S$, $(a + 1, b + 1) \in S$, and $(a + 2, b + 1) \in S$.

- List the elements of S produced by the first four applications of the recursive definition.
- Use strong induction on the number of applications of the recursive step of the definition to show that $a \leq 2b$ whenever $(a, b) \in S$.
- Use structural induction to show that $a \leq 2b$ whenever $(a, b) \in S$.

30. Give a recursive definition of each of these sets of ordered pairs of positive integers. [Hint: Plot the points in the set in the plane and look for lines containing points in the set.]

- $S = \{(a, b) \mid a \in \mathbb{Z}^+, b \in \mathbb{Z}^+, \text{ and } a + b \text{ is odd}\}$
- $S = \{(a, b) \mid a \in \mathbb{Z}^+, b \in \mathbb{Z}^+, \text{ and } a \mid b\}$
- $S = \{(a, b) \mid a \in \mathbb{Z}^+, b \in \mathbb{Z}^+, \text{ and } 3 \mid a + b\}$

- 31.** Give a recursive definition of each of these sets of ordered pairs of positive integers. Use structural induction

to prove that the recursive definition you found is correct. [Hint: To find a recursive definition, plot the points in the set in the plane and look for patterns.]

- a) $S = \{(a, b) \mid a \in \mathbf{Z}^+, b \in \mathbf{Z}^+, \text{ and } a + b \text{ is even}\}$
 - b) $S = \{(a, b) \mid a \in \mathbf{Z}^+, b \in \mathbf{Z}^+, \text{ and } a \text{ or } b \text{ is odd}\}$
 - c) $S = \{(a, b) \mid a \in \mathbf{Z}^+, b \in \mathbf{Z}^+, a + b \text{ is odd, and } 3 \mid b\}$
32. Prove that in a bit string, the string 01 occurs at most one more time than the string 10.
33. Define well-formed formulae of sets, variables representing sets, and operators from $\{\bar{}, \cup, \cap, -\}$.
34. a) Give a recursive definition of the function $\text{ones}(s)$, which counts the number of ones in a bit string s .
 b) Use structural induction to prove that $\text{ones}(st) = \text{ones}(s) + \text{ones}(t)$.
35. a) Give a recursive definition of the function $m(s)$, which equals the smallest digit in a nonempty string of decimal digits.
 b) Use structural induction to prove that $m(st) = \min(m(s), m(t))$.

The **reversal** of a string is the string consisting of the symbols of the string in reverse order. The reversal of the string w is denoted by w^R .

36. Find the reversal of the following bit strings.
 a) 0101 b) 11011 c) 100010010111
37. Give a recursive definition of the reversal of a string. [Hint: First define the reversal of the empty string. Then write a string w of length $n + 1$ as xy , where x is a string of length n , and express the reversal of w in terms of x^R and y .]
- *38. Use structural induction to prove that $(w_1 w_2)^R = w_2^R w_1^R$.
39. Give a recursive definition of w^i , where w is a string and i is a nonnegative integer. (Here w^i represents the concatenation of i copies of the string w .)
- *40. Give a recursive definition of the set of bit strings that are palindromes.
41. When does a string belong to the set A of bit strings defined recursively by
- $$\lambda \in A$$
- $$0x1 \in A \text{ if } x \in A,$$
- where λ is the empty string?
- *42. Recursively define the set of bit strings that have more zeros than ones.
43. Use Exercise 39 and mathematical induction to show that $l(w^i) = i \cdot l(w)$, where w is a string and i is a nonnegative integer.
- *44. Show that $(w^R)^i = (w^i)^R$ whenever w is a string and i is a nonnegative integer; that is, show that the i th power of the reversal of a string is the reversal of the i th power of the string.
45. Use structural induction to show that $n(T) \geq 2h(T) + 1$, where T is a full binary tree, $n(T)$ equals the number of vertices of T , and $h(T)$ is the height of T .

The set of leaves and the set of internal vertices of a full binary tree can be defined recursively.

Basis step: The root r is a leaf of the full binary tree with exactly one vertex r . This tree has no internal vertices.

Recursive step: The set of leaves of the tree $T = T_1 \cdot T_2$ is the union of the sets of leaves of T_1 and of T_2 . The internal vertices of T are the root r of T and the union of the set of internal vertices of T_1 and the set of internal vertices of T_2 .

46. Use structural induction to show that $l(T)$, the number of leaves of a full binary tree T , is 1 more than $i(T)$, the number of internal vertices of T .
47. Use generalized induction as was done in Example 13 to show that if $a_{m,n}$ is defined recursively by $a_{0,0} = 0$ and

$$a_{m,n} = \begin{cases} a_{m-1,n} + 1 & \text{if } n = 0 \text{ and } m > 0 \\ a_{m,n-1} + 1 & \text{if } n > 0, \end{cases}$$

then $a_{m,n} = m + n$ for all $(m, n) \in \mathbf{N} \times \mathbf{N}$.

48. Use generalized induction as was done in Example 13 to show that if $a_{m,n}$ is defined recursively by $a_{1,1} = 5$ and

$$a_{m,n} = \begin{cases} a_{m-1,n} + 2 & \text{if } n = 1 \text{ and } m > 1 \\ a_{m,n-1} + 2 & \text{if } n > 1, \end{cases}$$

then $a_{m,n} = 2(m + n) + 1$ for all $(m, n) \in \mathbf{Z}^+ \times \mathbf{Z}^+$.

- *49. A **partition** of a positive integer n is a way to write n as a sum of positive integers where the order of terms in the sum does not matter. For instance, $7 = 3 + 2 + 1 + 1$ is a partition of 7. Let P_m equal the number of different partitions of m , and let $P_{m,n}$ be the number of different ways to express m as the sum of positive integers not exceeding n .
- a) Show that $P_{m,m} = P_m$.
 b) Show that the following recursive definition for $P_{m,n}$ is correct:

$$P_{m,n} = \begin{cases} 1 & \text{if } m = 1 \\ 1 & \text{if } n = 1 \\ P_{m,m} & \text{if } m < n \\ 1 + P_{m,m-1} & \text{if } m = n > 1 \\ P_{m,n-1} + P_{m-n,n} & \text{if } m > n > 1. \end{cases}$$

- c) Find the number of partitions of 5 and of 6 using this recursive definition.



Consider the following inductive definition of a version of **Ackermann's function**. This function was named after Wilhelm Ackermann, a German mathematician who was a student of the great mathematician David Hilbert. Ackermann's function plays an important role in the theory of recursive functions and in the study of the complexity of certain algorithms involving set unions. (There are several different variants of this function. All are called Ackermann's function and have similar properties even though their values do not always agree.)

$$A(m, n) = \begin{cases} 2n & \text{if } m = 0 \\ 0 & \text{if } m \geq 1 \text{ and } n = 0 \\ 2 & \text{if } m \geq 1 \text{ and } n = 1 \\ A(m-1, A(m, n-1)) & \text{if } m \geq 1 \text{ and } n \geq 2 \end{cases}$$